

High-Level Design Document (HLDD)

Logistics E-Commerce & Order Management Web Application

Document ID:	HLDD-2026-LOG-001	Status:	Approved / Ready for Sprint Planning
Author:	Principal Solutions Architect	Date:	June 4, 2026
Repository:	GitHub Enterprise	PM Tool:	JIRA Software

1. Executive Summary

SUMMARY: This High-Level Design Document outlines the architectural patterns, functional frameworks, technological matrices, and deployment paradigms for the next-generation Enterprise Logistics Web Application. The core objective of this application is to deliver an end-to-end e-commerce interface optimized for B2B and B2C logistics management. Customers can natively browse catalog items, configure real-time shipping/freight orders, manage digital carts, and finalize secure transaction fulfillments via a state-of-the-art payment gateway ecosystem.

Engineered using a modern decoupled tier architecture, the solution leverages Angular for a responsive user interface, Node.js for scalable backend microservices, AWS for cloud-native infrastructure resilience, and Docker containerization to standardize environment configurations across development, staging, and production environments.

2. Functional Requirements

The system must support the following high-level operational flows to ensure an intuitive and streamlined client experience:

- **FR-101: Digital Catalog & Item Selection:** Users must be able to securely visit the web application, view a dynamically updated repository of available inventory items, filter based on logistics parameters, and select items for operational dispatch.
- **FR-102: Shopping Cart Subsystem:** Selected items must be capable of being staged within a transactional persistent shopping cart. The subsystem must track quantities, handle dynamic price computations based on shipping distances, and permit user updates (add, modify volume, remove) seamlessly.
- **FR-103: Secure Payment Gateway Integration:** The system must execute a seamless API handshake with an industry-standard external Payment Gateway (e.g., Stripe / Razorpay). It must guarantee PCI-DSS compliance, securely capturing, authorizing, and verifying transaction outcomes before finalizing orders.

- **FR-104: Order Generation & State Persistency:** Upon successful checkout and payment capture, a verifiable logistics order with an immutable tracking identifier must be instantly provisioned in the persistent database cluster.

3. Technology Stack

To support high concurrent availability, modularity, and smooth continuous delivery pipelines, the platform incorporates the following technologies:

Layer / Pillar	Technology Chosen	Architectural Justification & Role
Frontend UI Framework	Angular	Single Page Application (SPA) architecture providing structural cross-component consistency, strong TypeScript compilation, and robust state management for cart behaviors.
Backend Services API	Node.js / Express	Event-driven, asynchronous non-blocking I/O operations tailored to handle concurrent order volumes and rapid microservice communication pathways.
Containerization	Docker	Encapsulates runtime libraries, environment configurations, and microservices into light atomic images ensuring immutable deployments across localized nodes and AWS clusters.
Cloud Infrastructure	AWS	Enterprise-grade hosting utilizing Amazon ECS (Elastic Container Service) for container orchestration, Amazon RDS for secure data persistence, and AWS Secrets Manager for secure payment token storage.

4. High-Level Architecture & Integration

The system utilizes a 3-tier presentation, application, and infrastructure layer layout. Communication flows sequentially through the following decoupled milestones:

1. The client utilizes an **Angular** frontend layer to browse items and construct their logistics payloads.
2. Orders are pushed to the **Node.js** backend API cluster securely running within isolated **Docker** containers hosted inside an **AWS VPC**.
3. The backend microservices securely dispatch an API payload out to the **External Payment Gateway** via an encrypted TLS 1.3 channel.
4. Once an authorized success response callback is verified, the order updates its transactional ledger and triggers fulfillment tasks.

5. Project Management & Source Governance

To foster collaborative engineering velocity, agility, and absolute tracking transparency, the project strictly mandates the following tools:

- **Project Management Tool (JIRA):** The platform utilizes JIRA Software for tracking development velocity. Sprints will be organized into 2-week delivery increments. Epics, User Stories, and Bug tasks will map directly to the Functional Requirements defined in Section 2, ensuring rigorous traceability from concept to code deployment.
- **Version Control & Collaboration (GitHub):** All application source code will be securely provisioned and version-controlled inside GitHub Enterprise repositories. Mainline branches will enforce strict Branch Protection rules requiring pull request peer reviews and green status checks passing automated Docker compilation workflows.

6. Architectural Acceptance Criteria (AC)

A feature deployment is certified as structurally sound and operationally complete when it rigorously meets the following standards:

ID	Target Scope	Operational Validation Benchmark
AC-001	Catalog Browsing	The web application must load catalog lists containing up to 10,000 items in under 2.0 seconds over a standard 3G/4G network connection profile.
AC-002	Cart Persistency	Items moved to the cart must remain persistently allocated to the authenticated user's session profile during subsequent login cycles across multiple edge instances.
AC-003	Payment Gateway	All transactional exchanges must fully fail-safe; network timeouts during checkout must securely roll back the cart payload without executing double-debit anomalies.
AC-004	Infrastructure	The system must demonstrate consistent functional capability while running within the Docker cluster deployed inside the AWS cloud architecture.